

Sistema de Otimização de Rotas para Entregas Urbanas

Route Optimization System for Urban Deliveries

Alisson Silva, Ariel Liotto Angonese

Universidade Regional Integrada do Alto Uruguai e das Missões

Departamento de Engenharias e Ciência da Computação

Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

111585@aluno.uricer.edu.br, 111276@aluno.uricer.edu.br

Resumo. O crescimento dos serviços de entrega urbana e do comércio eletrônico tem intensificado a necessidade por soluções tecnológicas capazes de otimizar rotas e reduzir custos operacionais. Nesse contexto, este trabalho apresenta o desenvolvimento de um sistema web de roteirização voltado ao cálculo da rota mais eficiente entre um ponto inicial e múltiplos pontos de entrega em um ambiente urbano. A solução proposta utiliza dados geográficos reais obtidos a partir do OpenStreetMap, permitindo a representação da malha viária por meio de grafos ponderados. Para a determinação do caminho mais eficiente, é aplicado o algoritmo de Dijkstra, amplamente utilizado em problemas de caminho mínimo. O sistema integra conceitos de banco de dados, engenharia de software e teoria dos grafos, proporcionando uma aplicação funcional capaz de auxiliar no planejamento de rotas. Como resultado, espera-se demonstrar a viabilidade da utilização de algoritmos clássicos em conjunto com dados reais para a construção de soluções aplicadas à logística urbana.

Palavras-chave: Roteirização de Entregas; Grafos; Algoritmo de Dijkstra; OpenStreetMap; Otimização de Rotas.

Abstract. The growth of urban delivery services and e-commerce has increased the demand for technological solutions capable of optimizing routes and reducing operational costs. In this context, this work presents the development of a web-based routing system aimed at calculating the most efficient route between an initial point and multiple delivery points in an urban environment. The proposed solution uses real geographic data obtained from OpenStreetMap, allowing the representation of the road network through weighted graphs. To determine the most efficient path, Dijkstra's algorithm is applied, which is widely used in shortest path problems. The system integrates concepts from databases, software engineering, and graph theory, providing a functional application capable of supporting route planning. As a result, this work aims to demonstrate the feasibility of using classical algorithms combined with real-world data to build solutions applied to urban logistics.

Keywords: Delivery Routing; Graphs; Dijkstra's Algorithm; OpenStreetMap; Route Optimization.

1. Introdução

O crescimento das atividades de comércio eletrônico e dos serviços de entrega urbana tem ampliado significativamente a demanda por soluções tecnológicas capazes de otimizar deslocamentos dentro das cidades. Empresas de logística e entregadores independentes enfrentam diariamente o desafio de determinar rotas eficientes entre diferentes pontos urbanos, buscando reduzir o tempo de deslocamento, minimizar custos operacionais e aumentar a produtividade das operações. Entretanto, a definição manual de rotas pode ser imprecisa e ineficiente, principalmente em ambientes urbanos complexos, onde a rede viária apresenta grande quantidade de interseções, ruas e possíveis caminhos entre origem e destino.

Nesse contexto, técnicas computacionais baseadas na teoria dos grafos têm sido amplamente utilizadas para modelar redes de transporte e resolver problemas de roteirização. Ao representar interseções como nós e ruas como arestas, torna-se possível aplicar algoritmos de caminho mínimo para determinar rotas mais eficientes dentro de uma rede viária. Entre esses algoritmos, destaca-se o algoritmo de Dijkstra, amplamente utilizado para calcular o menor caminho entre dois pontos em grafos ponderados.

Paralelamente, a disponibilidade de bases de dados geográficas abertas tem facilitado o desenvolvimento de aplicações que utilizam informações reais sobre a estrutura urbana. Um exemplo relevante é o projeto OpenStreetMap, uma plataforma colaborativa que disponibiliza dados cartográficos detalhados de ruas, avenidas e interseções de diversas cidades ao redor do mundo. Esses dados permitem representar a malha viária de forma realista, possibilitando a construção de sistemas de roteirização aplicáveis a cenários urbanos reais.

Diante desse cenário, este trabalho propõe o desenvolvimento de um sistema web de roteirização voltado ao cálculo da rota mais eficiente entre um ponto de origem definido pelo usuário e múltiplos pontos de entrega. O sistema permite que o entregador informe o endereço ou as coordenadas geográficas do ponto de partida, defina os destinos desejados e visualize a rota otimizada calculada.

A problemática abordada neste trabalho está diretamente relacionada à otimização de rotas, tema amplamente estudado na literatura por meio do *Vehicle Routing Problem* (VRP), que envolve a definição de rotas eficientes para atendimento de múltiplos pontos. O sistema desenvolvido aborda esse problema aplicando o algoritmo de Dijkstra para calcular o menor caminho entre os pontos da rota, sem considerar, nesta versão, restrições operacionais como janelas de tempo ou capacidade de veículos.

Assim, o objetivo geral deste estudo é desenvolver um sistema capaz de calcular rotas eficientes em ambientes urbanos por meio da aplicação de algoritmos de caminho mínimo sobre dados reais de redes viárias. Como objetivos específicos, destacam-se a modelagem de um banco de dados para armazenamento das informações do sistema, a obtenção e estruturação de dados cartográficos, a implementação do algoritmo de Dijkstra para cálculo de rotas e a construção de uma interface que permita ao usuário definir um ponto de origem, adicionar

múltiplos pontos de entrega com seus respectivos destinatários e visualizar a rota otimizada calculada sobre o mapa real da cidade.

As principais contribuições deste projeto concentram-se na integração de diferentes áreas da computação, como bancos de dados, engenharia de software e teoria dos grafos, para a construção de um sistema funcional de roteirização urbana. Além de demonstrar a aplicação prática de algoritmos clássicos em problemas reais, o sistema também contribui para o entendimento de como dados geográficos e técnicas computacionais podem ser utilizados no desenvolvimento de soluções voltadas à otimização de processos logísticos.

2. Trabalhos Relacionados

O problema de roteirização de veículos, conhecido na literatura como *Vehicle Routing Problem* (VRP), é amplamente estudado nas áreas de otimização e logística. De forma geral, o VRP consiste em determinar rotas eficientes para atender um conjunto de pontos de entrega a partir de uma origem, minimizando custos como distância percorrida ou tempo de deslocamento. Esse problema possui diversas variações e aplicações práticas em sistemas de distribuição, transporte urbano e planejamento logístico, sendo considerado um dos problemas clássicos de otimização combinatória.

Entre as abordagens utilizadas para resolver problemas de roteirização, destacam-se os algoritmos de caminho mínimo em grafos. Um dos métodos mais conhecidos é o algoritmo de Dijkstra, que determina o menor caminho entre um vértice de origem e os demais vértices de um grafo ponderado. Esse algoritmo tornou-se base para diversos sistemas de navegação e planejamento de rotas utilizados em aplicações computacionais.

Diversos trabalhos exploram a aplicação de técnicas de otimização para resolver problemas de roteirização em cenários logísticos. Estudos investigam algoritmos aplicados a problemas de roteirização com restrições de janelas de tempo, demonstrando a relevância dessas técnicas em aplicações práticas de distribuição e logística.

Além disso, materiais acadêmicos disponibilizados por instituições como o *Massachusetts Institute of Technology* e a *Stanford University* apresentam fundamentos teóricos e aplicações de algoritmos de grafos em problemas de caminho mínimo e roteirização, evidenciando a importância dessas técnicas no desenvolvimento de sistemas computacionais voltados à otimização de rotas.

Nesse contexto, o presente trabalho utiliza o algoritmo de Dijkstra como base para o desenvolvimento de um sistema de roteirização voltado ao cálculo do menor caminho entre diferentes pontos de entrega. A proposta busca aplicar conceitos consolidados da teoria dos grafos em uma solução prática, permitindo a visualização e o planejamento de rotas de forma simples e acessível.

3. Fundamentação Teórica

3.1 HTML

O *HyperText Markup Language* (HTML) é uma linguagem de marcação utilizada para estruturar páginas na Web, permitindo a organização de conteúdos como textos, imagens, tabelas, formulários e links. Sua principal função é definir a estrutura lógica da informação, possibilitando que navegadores interpretem e exibam corretamente o conteúdo ao usuário.

O HTML foi desenvolvido por Tim Berners-Lee no início da década de 1990, com o objetivo de facilitar o compartilhamento de informações por meio da internet. Desde então, a linguagem passou por diversas evoluções, incorporando novos elementos e recursos que ampliaram suas funcionalidades e tornaram seu uso mais eficiente no desenvolvimento de aplicações web.

Com a padronização promovida pelo W3C, o HTML passou a ser amplamente adotado como base para o desenvolvimento de páginas web. Nesse contexto, o HTML5 representou um avanço significativo, introduzindo melhorias na semântica dos elementos e maior suporte à criação de aplicações interativas, além de reduzir a dependência de tecnologias externas.

No desenvolvimento web moderno, o HTML é responsável pela camada de estrutura da aplicação, organizando o conteúdo de forma lógica e semântica. Essa organização facilita não apenas a renderização nos navegadores, mas também a interpretação por mecanismos de busca e outros sistemas que consomem informações da web.

Além disso, o HTML atua em conjunto com outras tecnologias, como o CSS e o JavaScript, que são responsáveis, respectivamente, pela apresentação visual e pelo comportamento da aplicação. Dessa forma, o HTML constitui a base fundamental para o desenvolvimento de interfaces web, permitindo a construção de sistemas organizados, acessíveis e funcionais.

3.2 CSS

O *Cascading Style Sheets* (CSS) é uma linguagem utilizada para definir a apresentação visual de páginas web, incluindo aspectos como cores, fontes, espaçamentos e layout. O CSS permite adaptar a exibição do conteúdo para diferentes dispositivos, como computadores, dispositivos móveis e até meios impressos, sendo independente da linguagem de marcação utilizada.

No desenvolvimento web, o CSS desempenha o papel de separar a estrutura do conteúdo de sua apresentação visual. Por meio de regras de estilização, é possível aplicar características específicas a elementos HTML, tornando a interface mais organizada e visualmente atrativa. Uma regra CSS é composta por um seletor, que define o elemento a ser estilizado, e por declarações, que determinam as propriedades e valores aplicados.

A primeira versão do CSS foi proposta na década de 1990 e posteriormente padronizada pelo W3C, trazendo avanços significativos no controle visual das páginas web. Com o tempo,

novas versões foram desenvolvidas, ampliando as possibilidades de estilização e permitindo maior flexibilidade na criação de interfaces.

Com a evolução das necessidades do desenvolvimento web, o CSS passou a incorporar novos recursos, como seletores mais avançados, controle de layout e suporte a responsividade. Essas melhorias possibilitam a criação de páginas mais modernas e adaptáveis a diferentes contextos de uso.

Dessa forma, o CSS constitui uma tecnologia fundamental no desenvolvimento de interfaces web, permitindo a construção de aplicações visualmente organizadas e acessíveis. Sua integração com HTML e JavaScript possibilita a criação de sistemas completos, nos quais estrutura, apresentação e comportamento são tratados de forma independente.

3.3 JavaScript

O JavaScript é uma linguagem de programação amplamente utilizada no desenvolvimento web, sendo responsável por adicionar interatividade e comportamento às páginas. Atualmente, está presente na maioria dos sites modernos e é suportado por diversos dispositivos, incluindo computadores, smartphones, tablets e outros ambientes que possuem navegadores compatíveis.

No contexto do desenvolvimento web, o JavaScript atua em conjunto com o HTML e o CSS, formando as três principais camadas de uma aplicação. Enquanto o HTML define a estrutura e o conteúdo, e o CSS a apresentação visual, o JavaScript é responsável por controlar o comportamento dos elementos, permitindo a criação de interfaces dinâmicas e interativas.

A linguagem é classificada como interpretada, de alto nível, dinâmica e de tipagem flexível, o que proporciona maior adaptabilidade durante o desenvolvimento. Além disso, possui um conjunto de funcionalidades que constituem seu núcleo, permitindo a manipulação de dados e o controle do fluxo de execução de programas.

O JavaScript pode ser utilizado tanto no lado do cliente quanto no lado do servidor. No contexto *client-side*, a linguagem permite a manipulação do *Document Object Model* (DOM), possibilitando a alteração de elementos da página em tempo real, de acordo com a interação do usuário.

Já no contexto *server-side*, o JavaScript pode ser utilizado para o desenvolvimento de aplicações que executam no servidor, permitindo a realização de operações como acesso a banco de dados, manipulação de arquivos e comunicação com outros sistemas. Essa versatilidade contribui para que a linguagem seja amplamente utilizada em diferentes camadas de aplicações web.

3.4 Banco de Dados

Um sistema de banco de dados pode ser definido como um sistema computacional cujo objetivo é armazenar informações e permitir que usuários realizem operações como inserção,

consulta, atualização e remoção de dados. Essas informações podem representar elementos relevantes para indivíduos ou organizações, sendo utilizadas por aplicações específicas conforme suas necessidades.

Um sistema de gerenciamento de banco de dados (SGBD) é composto por uma coleção de dados inter-relacionados e por um conjunto de programas responsáveis por acessar e manipular esses dados. Seu principal objetivo é fornecer mecanismos eficientes de armazenamento e recuperação da informação, garantindo aspectos como integridade, segurança e consistência.

A base estrutural de um banco de dados está no modelo de dados, que consiste em um conjunto de conceitos utilizados para descrever a organização, os relacionamentos e as restrições dos dados. Esse modelo define como as informações são representadas e manipuladas dentro do sistema, permitindo uma interação estruturada entre o usuário e os dados.

Dentre os modelos existentes, destaca-se o modelo relacional, amplamente utilizado na prática. Nesse modelo, os dados são organizados em tabelas compostas por linhas e colunas, nas quais cada linha representa um registro e cada coluna um atributo. Essa estrutura facilita a organização, a consistência e a manipulação eficiente das informações.

Os sistemas de banco de dados também disponibilizam linguagens específicas para definição e manipulação dos dados. A linguagem de definição de dados (DDL) é utilizada para especificar a estrutura do banco, enquanto a linguagem de manipulação de dados (DML) permite realizar consultas e atualizações. Entre essas linguagens, destaca-se o SQL (*Structured Query Language*), amplamente utilizado em sistemas relacionais.

3.4.1 MySQL

O MySQL é um sistema gerenciador de banco de dados relacional amplamente utilizado, sendo considerado um dos SGBDs de código aberto mais populares. Sua ampla adoção está relacionada ao bom desempenho, confiabilidade e facilidade de utilização, características que o tornam uma escolha comum no desenvolvimento de aplicações web.

Baseado no modelo relacional e na linguagem SQL, o MySQL foi inicialmente voltado para aplicações de pequeno e médio porte. No entanto, com a evolução de suas versões, passou a ser utilizado também em sistemas que lidam com grandes volumes de dados, demonstrando sua capacidade de adaptação a diferentes cenários.

Além disso, o MySQL é distribuído sob a licença GNU/GPL, o que permite sua livre utilização, modificação e distribuição. Entre suas principais características, destacam-se a portabilidade, segurança e escalabilidade, bem como o suporte a diversas linguagens de programação, tornando-o uma solução versátil para o desenvolvimento de aplicações.

3.5 Python

Python é uma linguagem de programação de alto nível, interpretada e de propósito geral, amplamente utilizada no desenvolvimento de aplicações web, científicas e de automação. Sua

sintaxe simples e legível contribui para maior produtividade no desenvolvimento, sendo um dos fatores que explicam sua ampla adoção.

A linguagem foi criada por Guido van Rossum no final da década de 1980, com o objetivo de oferecer uma alternativa mais clara e eficiente em relação a outras linguagens. Desde então, Python evoluiu significativamente, incorporando recursos que facilitam o desenvolvimento de sistemas robustos e escaláveis.

Uma das principais características do Python é o suporte a múltiplos paradigmas de programação, incluindo programação orientada a objetos, funcional e procedural. Além disso, a linguagem possui uma vasta biblioteca padrão e um amplo ecossistema de bibliotecas externas, permitindo sua aplicação em diferentes áreas da computação.

No desenvolvimento web, Python é frequentemente utilizado no lado do servidor, sendo responsável pelo processamento de dados, lógica de negócio e comunicação com bancos de dados. Sua integração com diferentes tecnologias facilita a construção de sistemas completos e eficientes.

Outro fator relevante é a facilidade de aprendizado da linguagem, que, aliada à sua versatilidade, contribui para sua ampla adoção tanto em ambientes acadêmicos quanto profissionais. Dessa forma, Python se destaca como uma ferramenta adequada para o desenvolvimento de aplicações modernas.

3.5.1 Flask

Flask é um microframework web desenvolvido em Python, utilizado para a construção de aplicações web de forma simples e flexível. Ele fornece os componentes essenciais para o desenvolvimento, permitindo maior controle sobre a estrutura do sistema.

Diferentemente de frameworks mais robustos, o Flask adota uma abordagem minimalista, oferecendo suporte básico para roteamento, manipulação de requisições e geração de respostas. Essa característica torna o framework leve e adequado para aplicações de pequeno e médio porte.

Além disso, o Flask permite integração com diversas bibliotecas e extensões, possibilitando a adição de funcionalidades conforme a necessidade do projeto. Essa flexibilidade contribui para sua utilização em diferentes tipos de aplicações web, especialmente em sistemas que priorizam simplicidade e eficiência.

3.6 OpenStreetMap

O OpenStreetMap (OSM) é um projeto colaborativo que tem como objetivo a criação e disponibilização de dados geográficos livres e editáveis, permitindo que usuários ao redor do mundo contribuam para a construção de mapas digitais detalhados. A plataforma fornece informações sobre ruas, avenidas, interseções e diversos elementos geográficos, sendo amplamente utilizada em aplicações que dependem de dados cartográficos.

Uma das principais características do OpenStreetMap é seu modelo aberto, no qual qualquer usuário pode inserir, editar ou atualizar informações geográficas. Esse modelo colaborativo contribui para a constante atualização dos dados, tornando a plataforma uma alternativa relevante em relação a soluções proprietárias, especialmente em projetos acadêmicos e aplicações experimentais.

Os dados disponibilizados pelo OSM podem ser utilizados para representar redes viárias na forma de grafos, em que interseções são modeladas como vértices e as ruas como arestas. Essa estrutura é fundamental para a aplicação de algoritmos de roteirização, permitindo o cálculo de caminhos entre diferentes pontos com base em informações reais do ambiente urbano.

Além disso, o OpenStreetMap é amplamente integrado a bibliotecas e ferramentas computacionais que facilitam a manipulação e análise dos dados geográficos, possibilitando sua utilização em sistemas de navegação, planejamento urbano e logística. Essa integração contribui para o desenvolvimento de aplicações que utilizam dados reais de forma eficiente.

No contexto deste trabalho, os dados do OpenStreetMap são utilizados como base para a construção do grafo representando a rede de rotas. A partir dessas informações, torna-se possível aplicar algoritmos de caminho mínimo, como o algoritmo de Dijkstra, para determinar rotas eficientes entre pontos definidos pelo usuário.

3.7 Grafos

Um grafo é uma estrutura matemática utilizada para representar relações entre elementos, sendo composto por um conjunto de vértices (ou nós) e um conjunto de arestas que conectam esses vértices. Essa estrutura é amplamente empregada na computação para modelar problemas que envolvem conexões, como redes de transporte, comunicação e relações entre entidades.

Os grafos podem ser classificados de diferentes formas, como direcionados ou não direcionados, dependendo da existência de orientação nas arestas. Em grafos direcionados, as conexões possuem um sentido definido, enquanto em grafos não direcionados as conexões são bidirecionais. Essa distinção é fundamental para a modelagem adequada de diferentes tipos de problemas computacionais.

Além disso, grafos podem ser representados computacionalmente por meio de diferentes estruturas de dados, como matrizes de adjacência e listas de adjacência. A escolha da forma de representação influencia diretamente na eficiência dos algoritmos utilizados para percorrer e analisar o grafo, impactando no desempenho das soluções implementadas.

No contexto de sistemas computacionais, os grafos são amplamente aplicados na resolução de problemas que envolvem caminhos e conexões entre elementos. Sua utilização permite modelar cenários complexos de forma estruturada, facilitando a aplicação de algoritmos voltados à análise e otimização.

Dessa forma, a teoria dos grafos constitui uma base fundamental para o desenvolvimento de soluções computacionais que envolvem redes e roteirização, sendo essencial para a

compreensão de algoritmos utilizados em sistemas de navegação e logística.

3.7.1 Grafos Ponderados

Os grafos ponderados são uma variação dos grafos tradicionais em que cada aresta possui um valor associado, denominado peso. Esse peso pode representar diferentes métricas, como distância, tempo de deslocamento ou custo, dependendo da aplicação em que o grafo está sendo utilizado.

A utilização de pesos nas arestas permite a modelagem de problemas mais realistas, especialmente em cenários de transporte e logística, nos quais as conexões entre os pontos possuem custos distintos. Dessa forma, torna-se possível aplicar algoritmos que buscam otimizar esses valores, como no caso do cálculo do menor caminho.

Em aplicações de roteirização, os grafos ponderados são fundamentais, pois permitem representar a rede de caminhos de forma mais precisa. Isso possibilita a utilização de algoritmos específicos para determinar rotas eficientes, considerando os custos associados a cada trajeto.

3.8 Algoritmo de Dijkstra

O algoritmo de Dijkstra é um dos métodos mais conhecidos para a resolução do problema do caminho mínimo em grafos ponderados com pesos não negativos. Proposto por Edsger W. Dijkstra em 1959, esse algoritmo tem como objetivo determinar o menor custo para se deslocar de um vértice de origem até os demais vértices de um grafo.

O funcionamento do algoritmo baseia-se na exploração progressiva dos vértices do grafo, mantendo um conjunto de distâncias mínimas conhecidas a partir do vértice inicial. Inicialmente, a distância até o vértice de origem é definida como zero, enquanto as demais são consideradas infinitas. A cada iteração, o algoritmo seleciona o vértice com menor distância acumulada ainda não visitado, atualizando os valores de seus vizinhos sempre que um caminho mais curto é encontrado.

Esse processo, conhecido como relaxamento de arestas, é repetido até que todos os vértices tenham sido visitados ou até que o destino desejado seja alcançado. Como resultado, o algoritmo garante a obtenção do menor caminho possível entre os vértices, desde que os pesos das arestas sejam não negativos.

Do ponto de vista computacional, a eficiência do algoritmo depende da estrutura de dados utilizada em sua implementação. Em versões mais simples, sua complexidade é da ordem de $O(n^2)$, enquanto implementações mais otimizadas, utilizando filas de prioridade, podem alcançar complexidade de $O((V + E) \log V)$, onde V representa o número de vértices e E o número de arestas.

No contexto deste trabalho, o algoritmo de Dijkstra é utilizado para calcular a rota mais eficiente entre os pontos definidos pelo usuário em uma rede representada por um grafo ponderado. O algoritmo é aplicado de forma iterativa entre cada par consecutivo de pontos da rota, do

ponto inicial ao primeiro destino, do primeiro ao segundo, e assim sucessivamente, garantindo o menor caminho em cada trecho e contribuindo diretamente para a otimização de rotas em sistemas de entrega.

3.9 Roteirização de Entregas

A roteirização de entregas é um problema clássico na área de logística, que consiste em determinar rotas eficientes para a distribuição de produtos entre diferentes pontos. Esse problema está diretamente relacionado à otimização de recursos, como tempo e custo de deslocamento, sendo fundamental para empresas que atuam com transporte e distribuição.

Na literatura, esse tipo de problema é frequentemente associado ao *Vehicle Routing Problem* (VRP), que envolve a definição de rotas para veículos que devem atender um conjunto de clientes. O VRP apresenta diversas variações e restrições, como capacidade dos veículos, janelas de tempo e múltiplos depósitos, o que torna sua resolução um desafio computacional relevante.

Em cenários mais simplificados, a roteirização pode ser tratada como um problema de caminho mínimo em grafos, no qual se busca determinar a melhor rota entre dois pontos. Nesse contexto, a rede de transporte pode ser representada por um grafo, em que os vértices correspondem aos locais e as arestas representam os caminhos disponíveis, com custos associados.

A utilização de algoritmos de grafos, como o algoritmo de Dijkstra, permite calcular rotas eficientes considerando os custos definidos nas conexões entre os pontos. Essa abordagem é amplamente utilizada em sistemas de navegação e planejamento logístico, contribuindo para a redução de distâncias percorridas e otimização do tempo de entrega.

Dessa forma, a aplicação de técnicas de roteirização em sistemas computacionais possibilita melhorias significativas na eficiência operacional, sendo uma ferramenta importante para o apoio à tomada de decisão em processos logísticos e de distribuição.

4. Metodologia

A presente pesquisa possui caráter aplicado, uma vez que busca desenvolver uma solução computacional para o cálculo de rotas eficientes em um contexto de entregas urbanas. Quanto à abordagem, o trabalho é classificado como quantitativo, pois utiliza algoritmos e estruturas matemáticas para determinar o menor caminho entre os pontos de uma rede viária real. Em relação aos objetivos, trata-se de uma pesquisa de natureza exploratória e descritiva, voltada à aplicação de técnicas de otimização no desenvolvimento de um sistema de apoio ao planejamento de rotas.

O desenvolvimento do sistema baseou-se na modelagem da rede viária urbana como um grafo ponderado, no qual cada interseção é representada por um vértice e cada via por uma aresta com peso correspondente à sua extensão. O entregador informa o endereço ou as coordenadas geográficas do ponto de origem e define os pontos de entrega, informando o

endereço e os dados do destinatário de cada ponto. A partir dessa sequência, o algoritmo de Dijkstra é aplicado entre todos os pares de pontos relevantes, determinando a melhor ordem de visita de forma a minimizar a distância total percorrida. A rota completa é então apresentada ao entregador como o trajeto mais eficiente para o atendimento de todos os pontos selecionados.

Do ponto de vista tecnológico, o sistema foi desenvolvido utilizando Python com o microframework Flask para a construção do servidor web, HTML, CSS e JavaScript para a interface com o usuário, e MySQL para o armazenamento das informações. Os dados cartográficos foram obtidos a partir do OpenStreetMap e manipulados com o auxílio da biblioteca OSMnx, que representa a malha viária como um grafo ponderado. O cálculo das rotas é realizado por meio de uma implementação própria do algoritmo de Dijkstra, desenvolvida sem o uso de bibliotecas externas de grafos. Para a identificação do nó mais próximo no grafo em relação às coordenadas fornecidas pelo usuário, foi utilizada a distância euclidiana como métrica de aproximação, também implementada de forma manual.

Embora a fundamentação teórica apresente conceitos relacionados ao *Vehicle Routing Problem* (VRP) e suas variações — como restrições de janelas de tempo, capacidade de veículos e múltiplos depósitos —, tais aspectos não foram implementados nesta versão do sistema. O escopo atual está delimitado ao cálculo da rota mais eficiente entre um ponto de origem e os pontos de entrega selecionados pelo usuário, sem considerar restrições operacionais adicionais. No entanto, essas funcionalidades poderão ser incorporadas em versões futuras do sistema, ampliando sua aplicabilidade em cenários logísticos mais complexos.

O processo de desenvolvimento foi organizado em três etapas: planejamento, implementação e validação. Na etapa de planejamento, foram definidos o escopo do sistema, as tecnologias utilizadas e a modelagem do banco de dados. Na etapa de implementação, foram desenvolvidos o servidor web, a interface com o usuário, a integração com os dados do OpenStreetMap e o módulo de cálculo de rotas. Por fim, na etapa de validação, foram realizados testes com diferentes combinações de pontos de origem e destino, a fim de verificar o funcionamento do sistema e a consistência das rotas calculadas.

5. Análise dos Resultados

O sistema desenvolvido foi testado com diferentes combinações de pontos de origem e destino na cidade de Erechim, RS, utilizando dados reais da malha viária obtidos a partir do OpenStreetMap. Os testes realizados permitiram verificar o funcionamento do sistema em diferentes cenários, avaliando a consistência das rotas calculadas e o desempenho geral da aplicação.

A implementação do algoritmo de Dijkstra com fila de prioridade demonstrou eficiência satisfatória para o contexto urbano avaliado. O grafo da cidade de Erechim, composto por milhares de nós e arestas representando interseções e vias, foi carregado na inicialização do servidor e mantido em memória durante toda a execução, eliminando a necessidade de recarre-

gamento a cada requisição e contribuindo para a redução do tempo de resposta do sistema.

Os testes com múltiplos pontos de entrega confirmaram o funcionamento correto do cálculo sequencial de rotas, no qual o algoritmo de Dijkstra é aplicado entre cada par de pontos consecutivos e os segmentos resultantes são concatenados para formar o trajeto completo. As rotas calculadas respeitaram a malha viária real da cidade, seguindo ruas e avenidas existentes sem traçar caminhos em linha reta entre os pontos.

A interface web desenvolvida permitiu a visualização das rotas calculadas sobre o mapa real da cidade, utilizando dados do OpenStreetMap renderizados pela biblioteca Leaflet.js. O sistema também possibilitou o registro de entregas com múltiplos pontos e destinatários distintos e a consulta ao histórico de rotas anteriores.

Do ponto de vista da usabilidade, o sistema oferece duas formas de entrada de dados para os pontos da rota, por endereço textual, com geocodificação automática via Nominatim, ou por coordenadas geográficas inseridas manualmente, o que amplia sua aplicabilidade em situações onde o endereço não é reconhecido pela plataforma de geocodificação.

Os resultados obtidos demonstram a viabilidade da utilização de algoritmos clássicos de teoria dos grafos em conjunto com dados geográficos reais para o desenvolvimento de sistemas de roteirização urbana, confirmando os objetivos propostos no início do trabalho.

6. Considerações Finais

A presente pesquisa apresentou o desenvolvimento de um sistema web de roteirização voltado ao cálculo da rota mais eficiente entre um ponto de origem e múltiplos pontos de entrega em ambiente urbano. A solução proposta integrou conceitos de teoria dos grafos, banco de dados e engenharia de software, demonstrando a viabilidade da aplicação de algoritmos clássicos em conjunto com dados geográficos reais para a construção de sistemas de logística urbana.

O algoritmo de Dijkstra, implementado de forma manual com o uso de fila de prioridade, mostrou-se adequado para o contexto avaliado, calculando rotas eficientes sobre a malha viária real da cidade de Erechim a partir de dados obtidos do OpenStreetMap. A aplicação do algoritmo de forma sequencial entre pares de pontos consecutivos permitiu o suporte a múltiplos destinos em uma única rota, atendendo ao requisito central do sistema.

O sistema desenvolvido atingiu os objetivos propostos, oferecendo uma interface funcional para o planejamento e registro de entregas urbanas, com visualização das rotas calculadas sobre o mapa real da cidade. A integração entre frontend, backend e banco de dados possibilitou o funcionamento completo do fluxo de uma entrega, desde o cadastro até a consulta ao histórico de rotas anteriores.

O código-fonte completo do sistema, incluindo a documentação da API e o esquema do banco de dados, está disponível no repositório do projeto em: <https://www.forge.uricer.edu.br/2026-1-ProjetoIntegrador3/GrupoE>. O commit correspondente à entrega final pode ser acessado em: <http://forge.uricer.edu.br/2026-1-ProjetoIntegrador3/GrupoE>.

GrupoE/commit/699038054c48c6607f6205dc6ecb4dd7625ca3df.

Como trabalhos futuros, destacam-se as seguintes possibilidades de evolução do sistema:

- **Restrições de janelas de tempo** — incorporação de restrições temporais para o atendimento dos pontos de entrega dentro de intervalos definidos;
- **Capacidade de veículos** — limitação da quantidade de itens ou peso que cada veículo pode transportar por rota;
- **Múltiplos depósitos** — suporte a cenários com mais de um ponto de origem, permitindo a distribuição de entregas entre diferentes bases operacionais;
- **Otimização da ordem dos pontos** — implementação de heurísticas baseadas no problema do caixeiro viajante para determinar a sequência ótima de visita aos pontos de entrega.

Referências

DIJKSTRA, Edsger W. A Note on Two Problems in Connexion with Graphs. **Numerische Mathematik**, v. 1, p. 269–271, 1959.

FLASK DOCUMENTATION. Pallets Projects, 2024. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 29 mar. 2026.

LAPORTE, Gilbert. Fifty Years of Vehicle Routing. **Transportation Science**, v. 43, n. 4, p. 408–416, 2009.

MASSACHUSETTS INSTITUTE OF TECHNOLOGY. Introduction to Graph Theory - MIT OpenCourseWare. 2023. Disponível em: <https://ocw.mit.edu/>. Acesso em: 29 mar. 2026.

MASSACHUSETTS INSTITUTE OF TECHNOLOGY. Shortest Paths and Dijkstra’s Algorithm. 2023. Disponível em: <https://ocw.mit.edu/>. Acesso em: 29 mar. 2026.

MOZILLA DEVELOPER NETWORK. CSS: Cascading Style Sheets. 2024. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/CSS>. Acesso em: 29 mar. 2026.

MOZILLA DEVELOPER NETWORK. HTML: HyperText Markup Language. 2024. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTML>. Acesso em: 29 mar. 2026.

MOZILLA DEVELOPER NETWORK. JavaScript Guide. 2024. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. Acesso em: 29 mar. 2026.

MYSQL DOCUMENTATION. Oracle Corporation, 2024. Disponível em: <https://dev.mysql.com/doc/>. Acesso em: 29 mar. 2026.

OPENSTREETMAP CONTRIBUTORS. OpenStreetMap. 2024. Disponível em: <https://www.openstreetmap.org>. Acesso em: 29 mar. 2026.

PYTHON DOCUMENTATION. Python Software Foundation, 2024. Disponível em: <https://docs.python.org/3/>. Acesso em: 29 mar. 2026.

SOLOMON, Marius M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. **Operations Research**, v. 35, n. 2, p. 254–265, 1987.

STANFORD UNIVERSITY. Graph Algorithms: Shortest Paths. 2023. Disponível em: <https://web.stanford.edu/>. Acesso em: 29 mar. 2026.

STANFORD UNIVERSITY. Graph Theory and Algorithms Materials. 2023. Disponível em: <https://web.stanford.edu/>. Acesso em: 29 mar. 2026.

WORLD WIDE WEB CONSORTIUM. CSS: Cascading Style Sheets. 2024. Disponível em: <https://www.w3.org/Style/CSS/>. Acesso em: 29 mar. 2026.

WORLD WIDE WEB CONSORTIUM. HTML Standard. 2024. Disponível em: <https://html.spec.whatwg.org/>. Acesso em: 29 mar. 2026.